The Journal of Open Source Software

Torc: Durable and Resource-Aware Workflow Orchestration from Workstations to Slurm Clusters

Daniel Thom[1¶], **Nathan Keilbart**[2], **Joseph McKinsey**[1], **Pedro Andres Sanchez Perez**[1], **Andrew Lai**[2], **and Kapil Duwadi**[3]

1 National Laboratory of the Rockies **2** Lawrence Livermore National Laboratory **3** Pacific Northwest National Laboratory ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Computational studies require carefully prepared inputs across complex parameter sweeps, managing many simulations with varying computing requirements, and combining the results into a cohesive story all while tracking the provenance of inputs and outputs. Even with all this planning, calculations are sure to fail and should not require restarting an entire study.

Torc is an open-source workflow manager for analysts running existing simulation applications and scripts. Precompiled binaries support local use without a separate database service or a workflow programming language. On clusters managed by Slurm, Torc runs many jobs within each allocation of resources, selecting work that fits the available resources and remaining time rather than submitting every job separately. It retains searchable workflow history, results, and resource measurements. Recovery tools selectively repeat failed work and calculations affected by changed inputs; optional automated recovery retries memory- or time-limited jobs with increased resources. A common application programming interface supports command-line tools, an interactive terminal interface, a graphical web dashboard, and Python and Julia clients. Researchers can move from local studies to shared computational campaigns while preserving execution records and avoiding unnecessary repetition.

Statement of need

Scientific campaigns often begin with shell scripts or scheduler job arrays. These approaches can suffice for independent calculations, but large parameter studies combine dependent tasks with different CPU, GPU, and memory requirements. Analysts running long-lived studies on shared high-performance computing (HPC) systems need to inspect execution history, diagnose failures, and repeat only failed or invalidated work without wasting allocation time or becoming workflow engineers.

Meeting these needs requires more than submitting jobs: workflow state must persist beyond individual runs, and workers must keep available resources supplied with suitable work. Torc separates persistent workflow coordination from workers that claim jobs according to available resources and remaining allocation time. Execution, inspection, and recovery share one data model, allowing analysts to trace outcomes and repeat affected work without reconstructing a campaign from separate logs and scripts. Local operation requires no separately administered database service, while a shared server supports longer-lived campaigns and team access.

State of the field

Scientific workflow tools differ in how users define work, retain its history, and manage resources. Snakemake and Nextflow offer portable workflow languages (Di Tommaso et al., 2017; Mölder et al., 2021), while Parsl embeds parallel workflows in Python (Babuji et al., 2019). Pegasus combines workflow planning with provenance, data management, and recovery (Deelman et al., 2019). For persistent coordination, FireWorks and Balsam use services and workers (Jain et al., 2015; Salim et al., 2018), while AiiDA couples workflow automation with queryable data provenance (Huber et al., 2020). At the resource level, RADICAL-Pilot uses pilot jobs and Flux uses hierarchical scheduling to distribute tasks within allocated HPC resources. Both decouple application tasks from batch submissions, allowing one allocation to serve many tasks (Ahn et al., 2020; Merzky et al., 2019).

Torc's development team assessed existing workflow engines against the needs of analysts running established applications in large parameter studies. Local deployment needed precompiled binaries without separately managed services or language-specific runtime environments. Analysts needed easy to understand commands and configuration setup without learning a workflow programming language. On HPC systems, workers need to continually draw jobs from an external queue to maximize job allocations. Durable, queryable campaign history and integrated recovery were essential for managing repeated runs, while a language-independent API would support multiple interfaces and integration with other software.

Meeting these requirements led the team to separate persistent workflow state from resource-aware workers that claim jobs within allocations. The team judged that combining durable multi-user state with this execution model would require changes beyond an executor plugin in the evaluated Snakemake and Nextflow designs. FireWorks offered a closer architectural fit, but its MongoDB dependency conflicted with the goal of avoiding separately administered supporting services (Jain et al., 2015).

This is a fit-for-purpose rationale, not a claim that other systems cannot execute commands locally or on HPC resources. FireWorks and Balsam are particularly close architectural neighbors in their persistent, worker-based coordination. Torc's contribution is the integration of that operating model with standalone local use, SQLite-backed history, resource-aware execution, and common inspection and recovery tools. The design prioritizes these capabilities within one software package rather than requiring analysts to assemble a workflow engine, a database service, and separate operational tooling.

Software design

Torc uses a server to coordinate work and workers to execute it (Figure 1). The Rust server maintains the authoritative record of workflows, job dependencies, resource requests, and execution outcomes. Since the server has centralized database access and workers request jobs through its REST API, it can coordinate writes in transactions and batch job-completion updates. This makes lightweight SQLite practical without a separate database service, although its single-writer constraint still limits write throughput. Each worker reports its available resources and remaining runtime when requesting work, allowing the server to select ready jobs that fit. Workers execute those jobs, report outcomes, and request additional work as capacity becomes available. The server records these outcomes and releases jobs whose dependencies are satisfied.

The CLI can start a local server for a standalone command or connect to a long-running service for shared history and distributed execution. Standalone runs retain an on-disk database by default which can reside on the server's local filesystem rather than the cluster's parallel filesystem. On Unix, in-memory operation can improve performance by reducing filesystem

access, but risks losing changes since the last successful disk snapshot if the server or its hosting allocation is terminated unexpectedly or lost. Torc can save snapshots on demand or, in standalone mode, at configured intervals and before normal shutdown. Retained records can be queried by owner, project labels, or user-defined metadata. Database backups preserve these records, while external input files, outputs, and logs must be preserved separately.

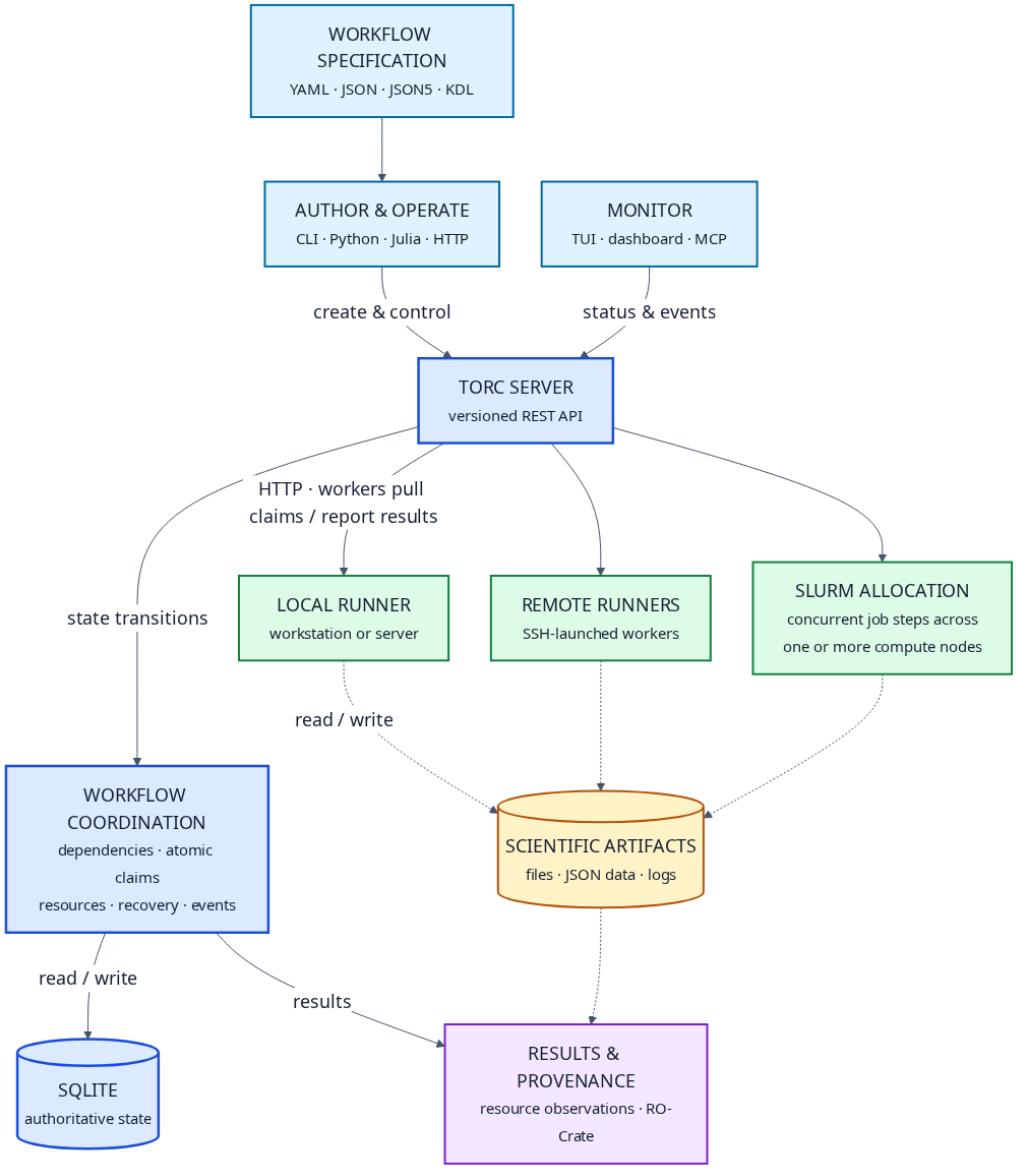


Figure 1: Torc separates user-facing tools, SQLite-backed workflow coordination, and pull-based execution across local, remote, and Slurm resources. Workers claim jobs and report outcomes over HTTP. Solid lines show control and state interactions; dashed lines show scientific artifact access.

Regardless of deployment mode, users describe the work to be coordinated in a workflow file containing commands, parameter ranges, dependencies, and resource requirements. Parameter expansion creates jobs for individual simulations, while explicit dependencies or input-output relationships connect preparation, simulation, and aggregation steps. Together, these definitions establish the dependency graph used to coordinate execution. The graph can also grow during execution: adaptive workflows add jobs transactionally, with lineage tracking and iteration limits. Existing scientific executables remain the unit of work, without

requiring their implementation in a workflow-specific programming interface.

Submitting the workflow registers its expanded jobs and dependencies with the server, and initialization identifies jobs that are ready to run. Once a worker comes online on a local or remote host, it requests ready jobs according to its available CPUs, memory, GPUs, node count, and remaining runtime. The server selects jobs and marks them pending in one transaction, preventing competing workers from claiming the same ready job. Priority ordering and backfill let ready CPU-only jobs use spare cores and memory alongside GPU jobs. Workers track the resources reserved by running jobs and release that capacity as jobs finish, including in direct execution mode. Within a Slurm allocation, workers repeat this process for the remaining walltime, avoiding a separate batch submission for each application job. As the scheduler handles allocation and placement while Torc selects dependency-ready work, this allows workers in multiple active allocations to draw jobs concurrently from the same workflow. Moving a workflow to a cluster preserves its dependency model but still requires compatible application environments, access to required files, server connectivity, and site-specific scheduler configuration.

As jobs finish, the server records their outcomes and updates dependency status, releasing blocked jobs once their prerequisites are satisfied. During temporary API outages, workers can journal completions locally and reconcile them with the server later. The resulting history of execution attempts helps users diagnose failures, while CPU, memory, and runtime measurements can guide subsequent resource requests. When monitoring and automatic recovery are enabled, client-side tools can identify likely memory or timeout failures, adjust resource requests, and retry selected jobs. For application-specific failures, users can define exit-code-based retry rules in the workflow file and optionally specify a recovery script to run before retrying a job.

Beyond recovering from failures, researchers may need to repeat a successful study with revised inputs or parameters. After applying definition changes to the stored workflow or modifying tracked input files, users reinitialize the workflow as a whole. Torc compares file modification times and job-input hashes with stored values, checks for missing outputs, and follows dependencies to identify which jobs need to run again. Affected jobs and their downstream dependents are reset for execution, while unaffected completed jobs are preserved. Reinitialization therefore checks the full workflow without requiring every job to be rerun.

Torc makes the same workflow records available for interactive use, programmatic control, and documentation of results. The CLI, interactive terminal interface, and web dashboard use a common REST API to inspect and manage workflows. An OpenAPI specification supports the generated Python and Julia clients included in the repository, as well as clients in other languages. The Model Context Protocol (MCP) server exposes workflow operations to AI assistants for inspection, debugging, and recovery, without making AI a requirement for execution or deterministic recovery. To document and share how results were produced, optional RO-Crate exports capture workflow, job, input, output, and software provenance from the recorded state (Soiland-Reyes et al., 2022).

Research impact statement

TODO before submission: Confirm which ExaEpi deployment details may be disclosed and what evidence can be shared with editors or reviewers. Obtain an approved reference or supporting record for the Dane PCM work; a research publication may not be available. Remove or qualify claims that cannot be supported, then remove this note.

At the National Energy Research Scientific Computing Center at Lawrence Berkeley National Laboratory, Torc orchestrated ExaEpi parameter sweeps for predicting epidemiological events, including COVID-19. The campaign ran three million simulations in batches of 100,000 on the Perlmutter system. Its Slurm workflow combined GPU-backed main simulations with

CPU-backed setup and postprocessing jobs. In the previous workflow, the delay before a worker picked up and started the next calculation could approach the runtime of a short simulation. Torc reduced this between-job dispatch delay, reducing campaign walltime rather than accelerating the simulations themselves. This experience illustrates the importance of keeping workers supplied with ready work when individual calculations are short.

At Lawrence Livermore National Laboratory's Livermore Computing Dane system, Torc ran power-system production-cost modeling (PCM) calculations across varied input scenarios. Each calculation modeled a year of power-system operation; this is the simulated period, not the execution duration. These CPU-only calculations ran longer than the ExaEpi simulations. Resource monitoring exposed CPU and memory use for debugging and allowed the team to tune resource requests for subsequent simulations. Together, these applications illustrate the value of dispatching short heterogeneous tasks and retaining resource evidence for longer-running analyses.

Availability

Torc is distributed under the BSD 3-Clause license. Source code, documentation, workflow examples, and precompiled binaries are available at https://github.com/NatLabRockies/torc.

TODO before publication: Archive the reviewed release, identify its version here, and add a citation and link to its persistent DOI. Remove this note once complete.

AI usage disclosure

OpenAI GPT-6 Astra Project, accessed through OpenCode, assisted with literature discovery, manuscript drafting and revision, figure preparation, formatting, and citation checks. Generative AI was also used in developing Torc. The authors made the software's core design decisions and retain responsibility for the software and manuscript.

Acknowledgements

Acknowledgements and funding information will be added before submission.

References

Ahn, D. H., Bass, N., Chu, A., Garlick, J., Grondona, M., Lipari, D., Martin, B., Stearman, M., Springmeyer, B., & Yang, T. (2020). Flux: Overcoming scheduling challenges for exascale workflows. *Future Generation Computer Systems*, *110*, 202–213. https://doi.org/10.1016/j.future.2020.04.006

Babuji, Y., Woodard, A., Li, Z., Katz, D. S., Clifford, B., Kumar, R., Lacinski, L., Chard, R., Wozniak, J. M., Foster, I., Wilde, M., & Chard, K. (2019). Parsl: Pervasive parallel programming in Python. *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 25–36. https://doi.org/10.1145/3307681.3325400

Deelman, E., Vahi, K., Rynge, M., Mayani, R., Silva, R. F. da, Papadimitriou, G., & Livny, M. (2019). The evolution of the Pegasus workflow management software. *Computing in Science & Engineering*, *21*(4), 22–36. https://doi.org/10.1109/MCSE.2019.2919690

Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, *35*(4), 316–319. https://doi.org/10.1038/nbt.3820

Huber, S. P., Zoupanos, S., Uhrin, M., Talirz, L., Kahle, L., Häuselmann, R., Gresch, D., Müller, T., Yakutovich, A. V., Andersen, C. W., Ramirez, F. F., Adorf, C. S., Gargiulo, F., Kumbhar, S., Passaro, E., Johnston, C., Merkys, A., Cepellotti, A., Mounet, N., … Pizzi, G. (2020). AiiDA 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance. *Scientific Data*, *7*, 300. https://doi.org/10.1038/s41597-020-00638-4

Jain, A., Ong, S. P., Chen, W., Medasani, B., Qu, X., Kocher, M., Brafman, M., Petretto, G., Rignanese, G.-M., Hautier, G., Gunter, D., & Persson, K. A. (2015). FireWorks: A dynamic workflow system designed for high-throughput applications. *Concurrency and Computation: Practice and Experience*, *27*(17), 5037–5059. https://doi.org/10.1002/cpe.3505

Merzky, A., Turilli, M., Maldonado, M., Santcroos, M., & Jha, S. (2019). Using pilot systems to execute many-task workloads on supercomputers. *Job Scheduling Strategies for Parallel Processing*, 61–82. https://doi.org/10.1007/978-3-030-10632-4_4

Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Lee, S., Twardziok, S. O., Kanitz, A., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., & Köster, J. (2021). Sustainable data analysis with Snakemake. *F1000Research*, *10*, 33. https://doi.org/10.12688/f1000research.29032.2

Salim, M. A., Uram, T. D., Childers, J. T., Balaprakash, P., Vishwanath, V., & Papka, M. E. (2018). Balsam: Automated scheduling and execution of dynamic, data-intensive HPC workflows. *Proceedings of the 8th Workshop on Python for High-Performance and Scientific Computing*.

Soiland-Reyes, S., Sefton, P., Crosas, M., Castro, L. J., Coppens, F., Fernández, J. M., Garijo, D., Grüning, B., La Rosa, M., Leo, S., Ó Carragáin, E., Portier, M., Trisovic, A., RO-Crate Community, Groth, P., & Goble, C. (2022). Packaging research artefacts with RO-Crate. *Data Science*, *5*(2), 97–138. https://doi.org/10.3233/DS-210053